

FoodRescue Developer Onboarding Guide

Welcome to the FoodRescue team! This guide will walk you through setting up the project on your local machine, how to run it, and the workflow for making and pushing your changes.

1. Local Setup

Step 1: Clone the Repository

First, clone the repository to your local machine.

```
git clone https://github.com/chingu-voyages/V61-tier3-team-32.git
cd V61-tier3-team-32
```

Step 2: Switch to the Development Branch

All our active work happens on the `development` branch. Switch to it immediately after cloning:

```
git checkout development
```

Step 3: Install Dependencies

This project is a monorepo containing both the frontend (`client`) and backend (`server`). We have a shortcut command that will install dependencies for everything at once:

```
npm run install:all
```

Step 4: Environment Variables

You need to set up environment variables for both the server and the root folder.

1. Copy the `.env.example` file located in the root of the project and rename the copy to `.env`.

```
cp .env.example .env
```

2. Open the new `.env` file and replace the placeholder values with the actual keys (Ask the team lead for the Supabase Database URL, JWT secret, and Cloudinary keys).

Step 5: Database Setup (Prisma)

We use Prisma to talk to our Supabase PostgreSQL database. To sync your local Prisma client with the database schema, run:

```
cd server
npx prisma generate
cd ..
```

(Note: Only run `npx prisma migrate dev` if you are actively making changes to the database schema in `schema.prisma`)

Step 6: Run the App!

To start both the React frontend and the Express backend simultaneously, run:

```
npm run dev
```

- **Frontend (Client)** will be available at: <http://localhost:5173>
- **Backend (Server)** will be available at: <http://localhost:5000>

2. Development Workflow (How to write & push code)

We use a strict branching strategy to prevent conflicts. **Never commit directly to `main` or `development`.**

Step 1: Ensure you are up to date

Before starting a new feature, make sure your local `development` branch has the latest code from GitHub:

```
git checkout development
git pull origin development
```

Step 2: Create a Feature or Fix Branch

Create a new branch off of `development` for the specific task you are working on.

- If it's a new feature, use `feature/`
- If it's a bug fix, use `fix/`

```
# Example for a new feature:
git checkout -b feature/login-page

# Example for a bug fix:
git checkout -b fix/header-alignment
```

Step 3: Write Code and Commit

Make your changes in your code editor. When you're ready to save your progress, add and commit your files:

```
# Add all changed files
git add .
```

```
# Write a clear, descriptive commit message
git commit -m "feat: added login form UI and validation"
```

Step 4: Push Your Branch to GitHub

When your feature is complete and working locally, push your branch up to the GitHub repository:

```
git push -u origin feature/login-page
```

Step 5: Open a Pull Request (PR)

1. Go to the repository on GitHub.
2. You will see a green button that says "**Compare & pull request**" next to your recently pushed branch. Click it!
3. **CRITICAL:** Ensure the "base branch" is set to `development` (NOT `main`).
4. Fill out a description of what you changed.
5. Click **Create pull request**.

Step 6: Get Approved and Merge

- Let the team know your PR is ready for review in Discord.
- You need at least **1 approval** from a teammate before you can merge.
- Once approved and all automated checks pass, you can click the "Merge pull request" button on GitHub to merge your code into `development` !

Troubleshooting

- **"PrismaClient is not defined"** -> You forgot to run `npx prisma generate` in the `server` folder.
- **"Connection refused" or API errors** -> Check your `.env` file and make sure the `DATABASE_URL` is correct.

- **"Port 5000 / 5173 is already in use"** -> You have another terminal window running the app. Close it or stop the process (Ctrl+C).

Happy Coding!